

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I V. Thanusri (192472265) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Thanusri

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent. Analyze the issue and explain how preprocessing improves feature detection performance.

Introduction

Feature detection is an important step in computer vision used to identify important points, edges, corners, blobs, and keypoints in an image. When images are captured under different lighting conditions, the same object may appear very different because of changes in brightness, contrast, shadows, reflections, and illumination. These variations can cause feature detectors to produce inconsistent results.

Problems caused by varying lighting conditions

1. Brightness variation: The same feature may have different intensity values in different images.
2. Low contrast: Important features may become difficult to distinguish from the background.
3. Shadows: Shadows can create false edges and corners.
4. Overexposure: Bright regions may lose useful details.
5. Underexposure: Dark regions may contain insufficient information.
6. Noise amplification: Poor lighting can increase sensor noise.
7. Different gradients: Edge and corner detectors depend on intensity changes, so lighting changes affect their responses.
8. False feature detection: Illumination changes may produce features that are not actually part of the object.

Role of preprocessing

Preprocessing modifies the image before feature detection so that important structures become more consistent.

1. Grayscale conversion

Color images can be converted into grayscale. A common conversion is:

$$\text{Gray} = 0.299R + 0.587G + 0.114B$$

This reduces image complexity and allows many feature detectors to work on intensity information.

2. Image normalization

Pixel intensities can be normalized to a standard range:

$$I_{\text{norm}} = (I - I_{\text{min}}) / (I_{\text{max}} - I_{\text{min}})$$

This reduces the effect of different exposure levels.

3. Histogram equalization

Histogram equalization improves contrast by redistributing intensity values. It is useful when an image is too dark or has poor contrast.

4. CLAHE

Contrast Limited Adaptive Histogram Equalization improves local contrast while limiting excessive noise amplification. It is useful when different areas of the image have different illumination.

5. Noise reduction

Gaussian, median, and bilateral filters can reduce unwanted noise. Gaussian smoothing is particularly useful before gradient-based feature detection.

6. Gamma correction

Gamma correction adjusts brightness:

$$I_{\text{out}} = I_{\text{in}}^{\gamma}$$

It can improve details in very dark or bright images.

7. Illumination correction

Uneven lighting can be reduced using background estimation or illumination normalization techniques.

Effect on feature detection

Without preprocessing:

Different lighting → Different intensity patterns → Different gradients → Inconsistent features

With preprocessing:

Lighting normalization → Stable intensity patterns → Better gradients → More consistent features

Example

Suppose a face-recognition system captures a person's face in bright sunlight, indoors, and in low light. Without preprocessing, the detector may identify different facial keypoints in each image. After normalization and contrast enhancement, important features such as eyes, nose corners, and mouth corners become more consistent.

Advantages

- Improves feature repeatability.
- Reduces noise.
- Enhances contrast.
- Reduces illumination effects.
- Improves edge and corner detection.
- Improves feature matching.
- Increases recognition accuracy.
- Reduces false feature detection.

Conclusion

Preprocessing is essential for reliable feature detection when images are captured under varying illumination. Techniques such as grayscale conversion, normalization, histogram equalization, CLAHE, filtering, and gamma correction reduce unwanted variations and make important image structures more stable. Therefore, preprocessing improves the accuracy, repeatability, and robustness of feature detection.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Introduction

Edge detection and corner detection are important techniques in computer vision, but they serve different purposes. Edge detection identifies boundaries where image intensity changes significantly, whereas corner detection identifies points where intensity changes significantly in multiple directions. For image matching, individual points are often more useful than long continuous edges.

Why edge detection is insufficient

An edge is generally a one-dimensional structure. For example, a straight boundary between two regions contains many points with similar characteristics. Therefore, it is difficult to determine a unique position along the edge.

Problems with edge detection for matching

1. Lack of uniqueness: Many points along a straight edge have similar characteristics.
2. Poor localization: An edge detector may identify an entire line rather than a specific point.
3. Ambiguity: It is difficult to determine which point on one edge corresponds to which point on another edge.
4. Sensitivity to noise: Small intensity changes can create unwanted edges.

5. Textured regions: Texture produces many edges, but these edges may not provide stable and distinctive matching points.
6. Geometric transformations: Matching edge pixels becomes difficult when an image is rotated or scaled.

Why corners are better

A corner is a point where intensity changes in two or more directions. It provides information in multiple directions and is therefore more distinctive.

Corner detection methods

- Harris Corner Detector
- Shi-Tomasi Corner Detector
- FAST
- Förstner detector

Harris Corner Detector

The Harris detector uses local image gradients. Its structure matrix is:

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

The Harris response is:

$$R = \det(M) - k(\text{trace}(M))^2$$

Interpretation:

- Flat region: Low response in both directions.
- Edge: High response in one direction.
- Corner: High response in both directions.

Comparison

Edge Detection:

- Detects boundaries.
- Usually produces continuous structures.
- Less distinctive.
- Difficult to match individual points.
- Useful for shape boundaries.

Corner Detection:

- Detects key points.
- Produces localized points.
- More distinctive.

- Easier to match.
- Useful for image matching.

Example

Consider a building image. Edge detection can identify roof edges, window boundaries, and wall boundaries. Corner detection can identify corners of windows, building corners, and intersections of lines. These points are more useful for matching the building across different images.

Why corners work better in textured regions

Textured regions contain many intensity variations. Corners represent locations where these variations occur in multiple directions. Therefore, corner points provide stronger local information than a single edge.

Feature matching pipeline

Input Images → Preprocessing → Corner Detection → Keypoint Description → Feature Matching → Object Recognition

Conclusion

Edge detection is useful for finding boundaries, but it does not always provide distinctive points required for feature matching. Corner detection identifies localized points with intensity changes in multiple directions, making them more distinctive and suitable for matching. Therefore, in images containing straight edges and textured regions, corner detection provides more useful keypoints than edge detection alone.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Introduction

The Hough Transform is a feature extraction technique used to detect geometric shapes such as lines, circles, and ellipses. For line detection, a line can be represented using the polar equation:

$$\rho = x \cos(\theta) + y \sin(\theta)$$

where ρ is the perpendicular distance from the origin and θ is the angle of the normal.

Effect of noise

Noise introduces unwanted intensity variations. After edge detection, these variations may appear as false edge pixels. These false edges vote in the Hough accumulator and create inaccurate results.

1. False lines: Noise pixels may accidentally form groups that vote for incorrect lines.

2. Weak actual lines: Noise can reduce the relative strength of the actual line.
3. Multiple peaks: The accumulator may contain many peaks corresponding to false lines.
4. Incorrect line orientation: Noise can produce votes for incorrect theta values.
5. Computational cost: A large number of false edge pixels increases voting operations.

Example

Suppose an image contains one actual road boundary. In a clean image, the Hough accumulator produces a strong peak for the actual line. In a noisy image, random pixels generate additional votes, producing several unwanted peaks.

Preprocessing improvements

1. Gaussian filtering: Reduces high-frequency noise before edge detection.
2. Median filtering: Effective against salt-and-pepper noise.
3. Bilateral filtering: Reduces noise while preserving important edges.
4. Morphological operations: Opening and closing can remove small unwanted structures and connect broken edges.

Improve edge detection

The quality of the Hough Transform depends strongly on the edge map. Canny edge detection can provide cleaner edges.

Improved pipeline:

Noisy Image → Gaussian/Median Filter → Canny Edge Detection → Hough Transform

Parameter selection

1. Threshold

A low threshold may produce many false lines, while a high threshold may miss weak actual lines. An appropriate threshold is required.

2. Theta resolution

Very fine angular resolution gives better accuracy but increases computational cost. Very coarse resolution is faster but less accurate.

3. Rho resolution

An appropriate distance resolution should be selected based on image size and required accuracy.

4. Minimum line length

In Probabilistic Hough Transform, minimum line length can be specified so short noise-generated segments are ignored.

5. Maximum line gap

This parameter allows broken parts of the same line to be connected.

Standard vs Probabilistic Hough Transform

Standard Hough:

- Uses many/all edge points.
- Higher computational cost.
- Detects lines globally.
- Can be strongly affected by noise.

Probabilistic Hough:

- Uses sampled edge points.
- Faster.
- Returns line segments.
- Can reduce unnecessary computation.

Improved pipeline

Noisy Image → Gaussian/Median Filtering → Canny Edge Detection → Morphological Cleaning → Hough Transform → Proper Threshold and Resolution → Detected Lines

Conclusion

Noise can significantly reduce Hough Transform accuracy because noisy pixels generate unwanted votes in parameter space. The problem can be reduced using Gaussian/median filtering, Canny edge detection, morphological operations, suitable accumulator thresholds, appropriate rho and theta resolutions, and minimum line-length constraints. Proper preprocessing and parameter selection improve both detection accuracy and computational efficiency.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Introduction

Feature matching identifies corresponding points between two images. Traditional feature matching may assume that images have similar scale, orientation, position, and appearance. When these assumptions are violated, matching performance decreases.

Problem with traditional feature matching

Suppose an object is captured at different sizes. The same feature may have completely different pixel sizes. If the object is rotated, the orientation of local gradients also changes. Traditional descriptors may therefore generate different feature representations.

Scale variation

Scale variation occurs when the object is closer to or farther from the camera. For example, one image may contain an object occupying 500×500 pixels while another contains the same object occupying only 200×200 pixels. A fixed-size feature window may not recognize the same feature.

Rotation variation

If an object is rotated by 90 degrees or another angle, its local gradient directions change. A rotation-sensitive descriptor may therefore produce different feature values.

Scale-invariant feature matching

Methods such as SIFT (Scale-Invariant Feature Transform) are designed to identify features that remain stable under scale and rotation changes.

Major steps of SIFT

1. Scale-space construction

The image is represented at multiple scales:

$$L(x,y,\sigma) = G(x,y,\sigma) * I(x,y)$$

2. Keypoint detection

SIFT detects stable keypoints across different scales using Difference of Gaussian:

$$D(x,y,\sigma) = L(x,y,k\sigma) - L(x,y,\sigma)$$

3. Keypoint localization

Unstable points and low-contrast points are removed.

4. Orientation assignment

A dominant orientation is assigned to each keypoint based on local image gradients. This helps make the descriptor rotation invariant.

5. Descriptor generation

A descriptor is created around each keypoint using local gradient information.

6. Feature matching

Descriptors from the two images are compared using a distance measure such as Euclidean distance.

Why scale-invariant methods work better

- Scale invariance: The same feature can be detected at different image sizes.
- Rotation invariance: The descriptor is aligned with the dominant orientation.
- Local representation: The descriptor represents the local neighborhood rather than the entire image.
- Robustness: It can tolerate moderate changes in scale, rotation, illumination, and viewpoint.

Comparison

Traditional Matching:

- Sensitive to scale.
- Sensitive to rotation.
- Fixed feature representation.
- Lower robustness.
- May produce incorrect matches.

Scale-Invariant Matching:

- Robust to scale changes.
- Robust to rotation.
- Multi-scale representation.
- Higher robustness.
- More reliable matching.

Example

Consider a logo recognition system where the same logo appears large in one image, small in another, and rotated in a third image. Traditional template matching may fail because the pixel patterns do not align. SIFT-like scale-invariant feature matching identifies stable keypoints and matches their descriptors despite scale and rotation changes.

Other robust methods

- SIFT
- SURF
- ORB
- AKAZE

Applications

Scale-invariant feature matching is used in object recognition, image stitching, panorama creation, robot navigation, augmented reality, 3D reconstruction, image retrieval, and landmark recognition.

Conclusion

Traditional feature matching is often sensitive to scale and orientation changes because corresponding features may have different sizes and orientations. Scale-invariant methods such as SIFT detect features across multiple scales and assign orientation information, producing descriptors that are more robust to geometric transformations. Therefore, they provide more accurate and reliable feature matching when images differ in scale and orientation.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy. Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Introduction

In computer vision and machine learning, an image can generate hundreds or thousands of features. Many of these features may be redundant, highly correlated, irrelevant, or computationally expensive. Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a smaller set of important components.

Problems with high-dimensional features

1. High computational cost: More features require more processing time.
2. High memory usage: Large feature vectors consume more memory.
3. Redundancy: Several features may contain similar information.
4. Curse of dimensionality: As dimensionality increases, learning algorithms may require more data and computation.
5. Overfitting: Irrelevant features can cause a model to learn noise rather than useful patterns.

How PCA works

PCA transforms correlated variables into a new set of uncorrelated variables called principal components. The components are ordered according to the amount of variance they explain.

Step 1: Construct feature matrix

$X = [x_1, x_2, \dots, x_n]$ for each sample.

Step 2: Standardization

Features are usually centered:

$$\mu_j = (1/m) \sum x_{ij}$$

$$X' = X - \mu$$

Standardization is especially important when features have different scales.

Step 3: Calculate covariance matrix

$$C = (1/(m-1)) X'^T X$$

The covariance matrix describes relationships between features.

Step 4: Calculate eigenvalues and eigenvectors

The covariance matrix is decomposed to obtain eigenvectors and eigenvalues. Eigenvectors represent principal directions, while eigenvalues indicate the amount of variance represented by each direction.

Step 5: Select principal components

Components are sorted according to eigenvalues. For example:

PC1 → 55% variance

PC2 → 25% variance

PC3 → 10% variance

PC4 → 5% variance

PC5 → 3% variance

PC6 → 2% variance

If the first three components explain 90% of the variance, the remaining components may be discarded depending on the required information retention.

Step 6: Transform the data

$$Z = X' W_k$$

where W_k contains the selected eigenvectors and Z is the reduced feature representation.

Example

Suppose a computer vision system extracts 1000 features. PCA may reduce these to 100 principal components while retaining most of the important information.

Benefits

1. Dimensionality reduction.
2. Reduced computational cost.
3. Reduced memory requirement.
4. Removal of redundancy.
5. Possible noise reduction.

6. Faster machine learning.
7. Easier visualization of high-dimensional data.

Before and after PCA

Without PCA:

Image → 1000 Features → Large Feature Vector → High Computation → Classification

With PCA:

Image → 1000 Features → PCA → 100 Important Components → Lower Computation → Classification

Limitations

1. PCA is a linear dimensionality reduction method.
2. Principal components may be difficult to interpret.
3. PCA does not necessarily select features that are best for classification.
4. It can be sensitive to feature scaling.
5. Some information may be lost when components are discarded.
6. PCA itself requires computational cost during training.

Applications

PCA is commonly used in face recognition, image compression, object recognition, pattern recognition, feature preprocessing, computer vision, data visualization, and machine learning.

Conclusion

PCA reduces a large feature set by transforming correlated features into a smaller number of uncorrelated principal components that preserve most of the important variance. By removing redundancy and low-information dimensions, PCA reduces computational cost, memory usage, and sometimes noise, making subsequent computer vision and machine learning tasks more efficient.

